

Privacy-themed token launch on Base

Whitepaper · Versione 0.1

Maggio 2026

Chain target: Base (Ethereum L2)

github.com/wayne97dev/nox

"Pay in the dark."

Versione: 0.1 · maggio 2026 **Repository:** github.com/wayne97dev/nox **Chain target:** Base (Ethereum L2) **Tagline:** *Pay in the dark.*

Abstract

Nox è un token ERC-20 deflazionario lanciato su Base attraverso una **vendita genesis a prezzo fisso** che termina con la **migrazione automatica a una pool Uniswap v4** dove ogni swap paga l'1% di fee al treasury via custom hook. Sopra al token vive un'**infrastruttura di pagamenti privati** basata su [ERC-5564](#) e [ERC-6538](#) (stealth addresses), e un **engine di mining a halving Bitcoin-style** che premia gli utenti che usano lo strumento di privacy. Il design è scelto per essere *legalmente difendibile*: nessun mixer custom, nessuna zk-circuitry proprietaria, nessuna logica che incentivi esplicitamente l'obfuscation. Tutto il sistema è immutabile post-seed: niente proxy upgradable, niente admin key, LP permanentemente bloccato.

1. Introduzione

1.1 Il problema

Le blockchain pubbliche sono *radicalmente trasparenti*: ogni saldo, ogni transazione, ogni interazione con uno smart contract è leggibile in chiaro per sempre. Questo è uno dei pochi mezzi di accountability che hanno gli utenti contro i progetti, ma è anche un grave problema di privacy individuale:

- Il salario in crypto è pubblico.
- I pagamenti tra privati sono pubblici.
- Lo stack di posizioni DeFi di chiunque è pubblico.
- Le donazioni a cause sensibili sono pubbliche.

I tentativi di risolvere questo problema sono storicamente di tre tipi:

1. **Privacy chain dedicate** (Monero, Zcash) — privacy forte ma fuori dall'ecosistema EVM.
2. **Mixer non-custodial** (Tornado Cash, Aztec) — privacy zk eccellente ma rischio regolatorio dimostrato: il caso Pertsev (condanna a 5 anni 4 mesi in Olanda, maggio 2024) ha stabilito un precedente penale concreto sul fatto di "scrivere codice di un mixer" indipendentemente dall'uso effettivo.
3. **Stealth addresses** (Umbra, Fluidkey) — privacy moderata del *receiver*, infrastruttura standardizzata da ERC, rischio legale praticamente nullo perché il pattern è uguale all'invio normale di un ERC-20 a un EOA.

Nox si posiziona deliberatamente sul terzo strato, costruendo un'esperienza utente nativa che renda l'uso degli stealth addresses la modalità *predefinita ed economicamente premiata* di muovere il token.

1.2 Perché Base + Uniswap v4

- **Base** ha gas bassi (median ~\$0.01 per swap), throughput alto, e una community DeFi consolidata.
- **Uniswap v4** introduce gli *hooks*, smart contract che possono iniettare logica prima/dopo ogni swap, add-liquidity, remove-liquidity. Permettono di sostituire la classica LP-fee con un meccanismo di fee custodito da un contratto esterno e direzionabile dove vogliamo.
- L'address-based hook system di v4 (i bit dell'indirizzo del hook determinano quali permessi ha) rende la logica del fee *verificabile staticamente* senza dover fidarsi di chi ha deployato.

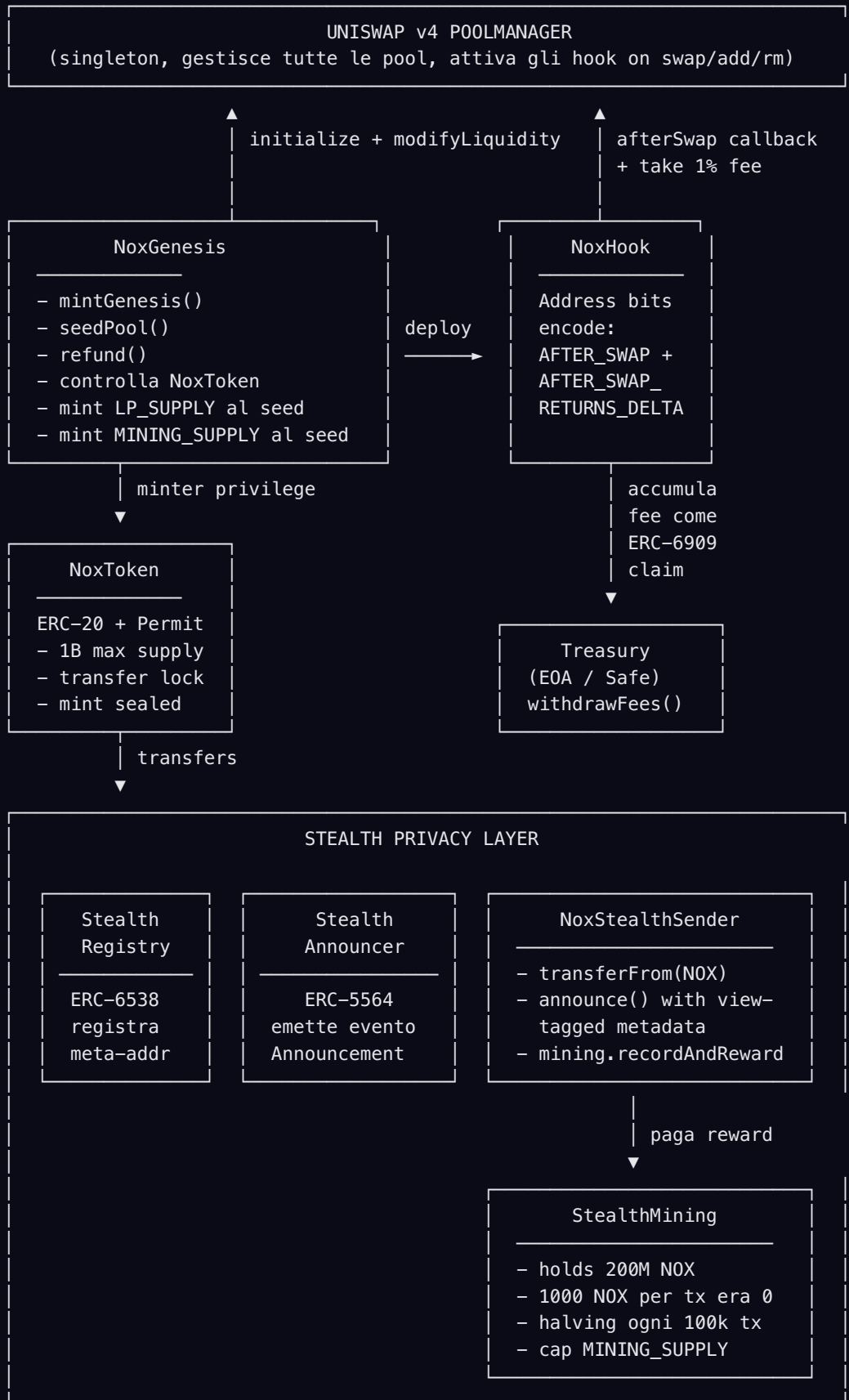
1.3 Cosa non è Nox

Per chiarezza:

- **Non è una privacy chain.** Le transazioni sono su Base in chiaro.
- **Non è un mixer.** Non rompiamo il link fra sender e amount.
- **Non usa zk-SNARKs.** Niente trusted setup, niente circuit Noir/Circom proprietari.
- **Non è una “privacy coin” come Monero.** È un ERC-20 normale con feature opzionali di privacy receiver-side.

2. Architettura del sistema

Nox è composto da **7 smart contract** + una **dApp web** + una **stealth crypto SDK** browser-side.



I 4 contratti del privacy layer sono interoperabili con qualunque wallet o protocollo conforme a ERC-5564, non solo con Nox. Il `NoxStealthSender` è invece NOX-specifico (lega l'invio al mining reward).

3. Il token NOX

3.1 Specifiche tecniche

Parametro	Valore
Standard	ERC-20 + ERC-2612 Permit
Nome	Nox
Simbolo	NOX
Decimals	18
Max supply	1,000,000,000
Mintable	Sì, ma solo da <code>NoxGenesis</code> ; sealable irreversibilmente
Burnable	No (per ora)
Pausable	No

Implementazione in `src/NoxToken.sol` . Eredita da `OpenZeppelin/ERC20` e `ERC20Permit` .

3.2 Transfer lock

Il token introduce un *transfer lock* che ricalca esattamente il design del riferimento Nonce sul quale il progetto si è ispirato:

```
function _update(address from, address to, uint256 value) internal override {
    if (!mintingClosed && from != address(0) && from != minter) {
        revert TransfersLocked();
    }
    super._update(from, to, value);
}
```

Tradotto: **prima del `seedPool()` finale, NOX è completamente intransferibile** fra utenti. Gli unici movimenti permessi sono:

- **Mint** (`from == address(0)`): permette al `NoxGenesis` di emettere token ai buyer durante la genesis.
- **Transfer dal minter stesso** (`from == minter`): permette al `NoxGenesis` di spedire i 200M di LP al PoolManager durante il seed.

Una volta che `NoxGenesis` chiama `sealMinting()` (in coda a `seedPool()`), `mintingClosed` diventa `true` per sempre e il check sopra non blocca più nulla. Da quel momento NOX è un normale ERC-20.

Conseguenza per il buyer: durante la genesis non puoi vendere, regalare, o spostare i tuoi NOX. La tua unica via di uscita prima del seed è il rimborso (vedi §4.4).

3.3 Tokenomics

Allocazione	Quantità	%	Destinazione
Genesis sale	600,000,000	60%	Buyer pubblici a 0.00001 ETH per 1,000 NOX
Liquidity pool	200,000,000	20%	Seeded full-range nella v4 pool, LP locked forever
Stealth mining	200,000,000	20%	Emessi via <code>StealthMining</code> con halving
Totale	1,000,000,000	100%	

Nessuna allocazione team/insider. Nessuna allocazione marketing. Nessuna allocazione “treasury” pre-mint. Il treasury si forma esclusivamente dall’1% di fee sugli swap post-seed.

4. Genesis Sale

4.1 Modello

Vendita a **prezzo fisso** per un periodo fisso (“window”), con cap massimo. Non è una bonding curve nel senso classico (no formula $x*y=k$, no prezzo che cresce con la domanda): è più simile a un **bond fundraise** in cui ogni token ha lo stesso costo per tutti i buyer.

Parametro	Valore	Note
<code>GENESIS_PRICE</code>	0.00001 ETH	per ogni unità di acquisto
<code>GENESIS_UNIT</code>	1,000 NOX	quanto NOX riceve una unità
<code>GENESIS_CAP_UNITS</code>	600,000	unità totali in vendita
Implied raise	6 ETH	cap pieno = 6 ETH raccolti
<code>MAX_UNITS_PER_TX</code>	10,000	= 10M NOX per transazione, ~1.7% del cap
<code>MAX_MINTS_PER_BLOCK</code>	5	anti block-stuffing
Window	configurabile al deploy (default 7 giorni)	
<code>REFUND_GRACE</code>	48 ore	dopo close window se nessun seed

4.2 Anti-bot

Due meccanismi semplici ma efficaci:

1. **Per-tx cap:** `MAX_UNITS_PER_TX = 10,000` impedisce ad un singolo buyer di prendere tutto il cap in una transazione. Nei fatti, riempire il cap richiede minimo 60 transazioni.
2. **Per-block cap:** `MAX_MINTS_PER_BLOCK = 5` impedisce a un bot di occupare l'intero blocco con `mintGenesis` calls. Su Base un blocco è 2 secondi, quindi al massimo 5 mint ogni 2s.

Combinato: il cap minimo di tempo per saturare il sale è $\approx 60 / 5 \times 2s = \mathbf{24 \text{ secondi}}$, ma realisticamente molto più dato che diversi utenti competono per gli slot per blocco.

4.3 Sicurezza del raise

Tutto l'ETH ricevuto in `mintGenesis` rimane nel `NoxGenesis` contract fino a `seedPool()`. **Non c'è alcuna funzione per ritirare l'ETH al di fuori del seed o del refund.** Né l'owner, né il controller, né nessun altro può estrarre i fondi.

Il `controller` (un EOA configurato al deploy) ha esattamente **un solo potere**: chiamare `seedPool()` *anche se il cap non è raggiunto* — ma solo dopo che la window è scaduta. Tutti gli ETH raccolti vanno comunque nella LP. Non c'è modo di rugpullare.

4.4 Refund fallback

Se per qualunque motivo `seedPool()` non viene chiamato (per esempio: il cap non si riempie e il controller decide di non forzare il seed perché il raise è troppo piccolo), allora **48 ore dopo la fine della window** ogni buyer può chiamare:

```
function refund() external {
    if (seeded) revert AlreadySeeded();
    if (block.timestamp < closeAt + REFUND_GRACE) revert GraceNotPassed();
    uint256 amount = ethPaid[msg.sender];
    if (amount == 0) revert NothingToRefund();
    ethPaid[msg.sender] = 0;
    (bool ok,) = msg.sender.call{value: amount}("");
    require(ok, "refund failed");
}
```

Il buyer recupera **1 ETH per 1 ETH versato**. I NOX nel suo wallet restano lì ma sono per sempre in-transferibili (transfer lock attivo perché `mintingClosed` non sarà mai `true`).

4.5 Stati possibili al termine della window

Stato finale	Chi può chiamare <code>seed-Pool()</code>	Refund possibile?
Cap raggiunto in qualsiasi momento	Chiunque (gas-payer)	No
Window scade con cap non raggiunto	Solo controller	Solo se non si chiama → dopo grace
Controller non interviene + 48h	Nessuno (window chiusa)	Sì, buyer 1:1

5. Pool seed su Uniswap v4

5.1 Architettura v4

A differenza di v2/v3 dove ogni pool è un contratto, **Uniswap v4 ha un singolo `PoolManager` singleton** che gestisce tutte le pool. Una pool è identificata da una `PoolKey` :

```
struct PoolKey {
    Currency currency0; // token0 address (o 0x0 per native ETH)
    Currency currency1; // token1 address
    uint24 fee; // LP fee in 1e6-bp (0.3% = 3000)
    int24 tickSpacing; // granularità ticks
    IHooks hooks; // hook contract
}
```

Per noi:

Campo	Valore	Perché
<code>currency0</code>	<code>0x0</code>	Native ETH
<code>currency1</code>	<code>address(NoxToken)</code>	NOX
<code>fee</code>	<code>0</code>	No LP fee — la fee è 100% nel hook
<code>tickSpacing</code>	<code>60</code>	Standard per pool 0.3%-equivalente
<code>hooks</code>	<code>address(NoxHook)</code>	Custom 1% fee hook

5.2 Inizializzazione della pool

`NoxGenesis.seedPool()` esegue, in atomico:

1. Calcola `sqrPriceX96` iniziale come $\sqrt{LP_SUPPLY / ethRaised} \times 2^{96}$. Con cap pieno: $\sqrt{200,000,000e18 / 6e18} \times 2^{96} \approx 5773.5 \times 2^{96}$.
2. Chiama `poolManager.initialize(poolKey, sqrPriceX96)` — crea la pool al prezzo iniziale.
3. Calcola la quantità massima di liquidità depositabile full-range con i fondi raccolti via `LiquidityAmounts.getLiquidityForAmounts`.
4. Chiama `poolManager.unlock(...)` per entrare nel contesto unlocked (in v4 ogni manipolazione di stato richiede unlock).
5. Dentro `unlockCallback`: a. `poolManager.modifyLiquidity(key, params, "")` — registra la posizione full-range a nome di `NoxGenesis`. b. `poolManager.settle{value: eth0wed}()` — paga ETH al PoolManager. c. `token.mint(this, nox0wed)` — minta l'esatta quantità di NOX necessaria (rounding-aware). d. `poolManager.sync(noxCurrency)` + `token.transfer(poolManager, nox0wed)` + `poolManager.settle()` — paga NOX al PoolManager.
6. `token.sealMinting()` — chiude per sempre il mint. Da questo momento `MAX_SUPPLY` è raggiunto e immutabile.

5.3 LP locked forever

La posizione di liquidità è registrata al PoolManager come appartenente a `NoxGenesis`. **NoxGenesis non espone nessuna funzione di `removeLiquidity`**. Codice completo dell'interfaccia esterna:

```
function mintGenesis(uint256 units) external payable;
function seedPool() external;
function refund() external;
function unlockCallback(bytes calldata data) external returns (bytes memory);
// + getter pubblici per gli storage
```

Nessuna voce “withdraw”. Nessuna “migrate”. Nessuna “emergency”. Nessuna funzione `onlyOwner`. La liquidità nella pool v4 è **letteralmente prigioniera** del contratto fino alla fine dell'universo.

5.4 NoxHook: la fee dell'1%

NoxHook è un hook v4 che implementa solo **afterSwap**. È deployato con CREATE2 a un indirizzo i cui ultimi 14 bit codificano i flag **AFTER_SWAP_FLAG** | **AFTER_SWAP_RETURNS_DELTA_FLAG**. Il **HookMiner** brute-forza il salt necessario.

Per ogni swap il hook:

1. Identifica il token “unspecified” dello swap (quello dell’output side).
2. Calcola $\text{feeAmount} = \text{output} \times 100 / 10000 = 1\%$.
3. Chiama `poolManager.mint(address(this), currency.toId(), feeAmount)` — accumula la fee come *claim* ERC-6909 contro il PoolManager (non sposta token fisicamente, è più economico).
4. Ritorna come “delta” per riequilibrare i conti del PoolManager.

La fee si accumula nel tempo. Periodicamente *chiunque* può chiamare:

```
function withdrawFees(Currency currency) external returns (uint256 amount);
```

che brucia il claim ERC-6909 e trasferisce la quantità accumulata al **treasury** immutabile configurato al deploy. Il treasury è un singolo indirizzo (EOA, Safe multisig, o splitter contract). Non c’è privilegio: chiunque paga gas può triggerare il withdraw, ma i fondi vanno sempre al treasury.

6. Privacy Layer: Stealth Addresses (ERC-5564)

6.1 Il modello hawala-on-chain

L’analogia che il progetto vuole catturare è il *hawala*, sistema di rimesse informale dove due agenti remoti accreditano simultaneamente importi a clienti diversi senza che il denaro fisicamente attraversi confini. Il “collegamento” tra le due tratte è un *codice* (una ricevuta) che il cliente porta da una parte all’altra.

Stealth addresses portano lo stesso pattern on-chain con una differenza cruciale: il “**codice**” è **crittografico**, non un nome scritto. Senza la chiave segreta, nessuno può collegare due transazioni anche conoscendo *tutti* gli indirizzi coinvolti.

6.2 Schema crittografico (scheme 0 — SECP256k1)

Il receiver ha **due chiavi private** distinte:

- **k_s** (*spending key*) — controlla la spesa dei fondi
- **k_v** (*viewing key*) — usata solo per *scoprire* i pagamenti ricevuti

Dalle private discendono le pubbliche $K_s = k_s \cdot G$ e $K_v = k_v \cdot G$ (G è il generatore di SECP256k1).

La **meta-address** pubblicata on-chain è la concatenazione:

```
meta_address = K_s || K_v (66 byte, due chiavi compresse da 33 byte)
```

Lato sender

Per pagare un receiver di cui conosce la meta-address:

1. Genera una chiave effimera random r (32 byte).
2. Calcola $R = r \cdot G$ (chiave pubblica effimera, 33 byte compressi).
3. Calcola il punto condiviso $S = r \cdot K_v$ (ECDH con la viewing key del receiver).
4. Calcola $h_s = \text{keccak256}(S.x)$ dove $S.x$ è la coordinata X di S (32 byte).
5. Calcola la chiave pubblica stealth: $P = K_s + h_s \cdot G$.
6. Calcola l'indirizzo Ethereum di P (keccak256 di X||Y, ultimi 20 byte) → `stealth_address`.
7. Estrae il **view tag** = primo byte di h_s (per filtraggio veloce lato receiver).

Tutto questo avviene **off-chain nel browser del sender** (vedi `lib/stealth.ts`).

Poi il sender chiama `NoxStealthSender.sendStealthNox(0, stealth_address, amount, R, viewTag)` che on-chain:

```
nox.transferFrom(msg.sender, stealthAddress, amount);
announcer.announce(schemeId, stealthAddress, ephemeralPubKey, metadata);
reward = mining.recordAndReward(msg.sender);
```

Il metadata segue il formato Umbra:

```
metadata = viewTag (1 byte) || tokenAddress (20 byte) || amount (32 byte)
```

Lato receiver

Per scoprire i pagamenti ricevuti, il receiver scansiona tutti gli `Announcement` events emessi da `StealthAnnouncer`. Per ogni evento:

1. Estrae R (ephemeralPubKey) e `metadata`.
2. Calcola $S = k_v \cdot R$ (lo stesso punto del sender per la proprietà ECDH $r \cdot K_v = k_v \cdot R$).
3. Calcola $h_s = \text{keccak256}(S.x)$ e ne estrae il view tag (primo byte).
4. **Fast filter:** confronta il view tag computato con `metadata[0]`. Se non coincide, scarta. Questo step elimina ~99.6% degli eventi senza fare crittografia pesante (1 byte = 256 possibilità).
5. Se il view tag coincide, ricalcola $P = K_s + h_s \cdot G$ e ne deriva l'address. Confronta con `stealthAddress` dell'evento. Se coincide è un pagamento per noi.
6. Estrae `amount` e `token` dal metadata.
7. **Deriva la stealth private key** per spendere i fondi: $p = (k_s + h_s) \bmod n$, dove n è l'ordine della curva. Questa chiave controlla `stealth_address` ed è importabile in qualunque wallet.

Tutto il flusso receiver-side vive in `lib/stealth.ts` (browser) ed è esposto dalla pagina `/stealth/receive` della dApp.

6.3 Cosa è privato e cosa no

Tabella onesta:

	Sender on-chain	Receiver on-chain	Amount	View tag	Link sender→receiver
Trasferimento ERC-20 standard	✓ pubblico	✓ pubblico	✓ pubblico	—	✓ banale
Nox stealth send	✓ pubblico (sender main wallet)	✗ pseudonimo (one-time addr)	✓ pubblico	✓ pubblico	✗ rotto

Cosa restano pubblici: - L'indirizzo del sender che chiama `NoxStealthSender` (per il chain-watcher è "Alice ha pagato X NOX a un indirizzo stealth"). - L'amount del pagamento. - L'indirizzo stealth one-time (utile solo per il receiver).

Cosa diventa privato: - L'identità del receiver. Nessuno, guardando l'indirizzo stealth, può capire a chi appartiene fra i registranti del registry. Per scoprirlo dovrebbe risolvere il problema della *Decisional Diffie-Hellman*, intrattabile per chiunque non abbia la viewing key del receiver.

Questa è privacy **del receiver**, non del sender né dell'amount. Per privacy completa serve un mixer/shielded pool (vedi §10).

6.4 Implementazione

Contract	Standard	Linee	Dove
<code>StealthRegistry</code>	ERC-6538	56	<code>src/stealth/StealthRegistry.sol</code>
<code>StealthAnnouncer</code>	ERC-5564	27	<code>src/stealth/StealthAnnouncer.sol</code>
<code>NoxStealthSender</code>	custom	56	<code>src/stealth/NoxStealthSender.sol</code>
Stealth SDK browser	—	280	<code>web/lib/stealth.ts</code>

`StealthRegistry` e `StealthAnnouncer` sono **standard generici, non NOX-specifici**. Possono essere riusati per qualunque ERC-20 e sono compatibili con wallet/SDK conformi a ERC-5564 (Umbra, Fluidkey, Squid).

7. Stealth Mining

7.1 Idea

Premiare l'uso *non-sensibile* della privacy. Ogni volta che un utente paga via `NoxStealthSender`, lui (il sender) riceve una piccola quantità di NOX come reward. Più alta è l'attività di pagamenti privati nel network, più velocemente si distribuisce la mining supply.

Il design è ispirato al **modello Bitcoin di halving**, applicato però ad un contatore di transazioni anziché di blocchi:

Parametro	Valore	Note
<code>MINING_SUPPLY</code>	200,000,000 NOX	20% del total supply
<code>INITIAL_REWARD</code>	1,000 NOX	reward al sender per la prima tx dell'era 0
<code>ERA_TX_COUNT</code>	100,000	numero di tx dopo cui la reward dimezza
Reward in era n	$INITIAL_REWARD / 2^n$	halving
Emission cap	<code>MINING_SUPPLY</code>	la reward si azzerà quando il cap è raggiunto

7.2 Emission schedule

Era	Reward / tx	Tx range	NOX emessi nell'era	Cumulativo
0	1,000	0–99,999	100,000,000	100,000,000
1	500	100,000–199,999	50,000,000	150,000,000
2	250	200,000–299,999	25,000,000	175,000,000
3	125	300,000–399,999	12,500,000	187,500,000
4	62.5	400,000–499,999	6,250,000	193,750,000
...	...	...	...	...
$n \rightarrow \infty$	$\rightarrow 0$	—	$\rightarrow 0$	$\rightarrow 200,000,000$

La serie converge geometricamente a $INITIAL_REWARD \times ERA_TX_COUNT \times 2 = 200,000,000$, esattamente `MINING_SUPPLY`. Il contratto include comunque un hard cap che azzerà la reward se per qualunque motivo si arrivasse a sfiorare.

7.3 Perché premiare l'uso degli stealth e non altre attività

Discussione delle alternative considerate:

Cosa premia il mining	Pro	Contro	Verdetto
Uso di un miner	Forte incentivo a privacy seria	Caso Pertsev: incentivare l'obfuscation = aggravante penale per i dev	❌ Scartato
Volume di swap sulla v4 pool	Standard DeFi	Wash-trading farmable senza limiti	❌ Sub-ottimali
LP staking sulla v4 pool	Standard DeFi	Va in concorrenza con il LP-fee model che abbiamo a 0%	⚠️ Possibile ma duplicato
Holding shielded (Railgun)	Forte privacy	Dipendenza da protocollo esterno + visibilità "incentivo allo shielding"	⚠️ Possibile ma rischio narrativo
Uso di stealth addresses	Standard ERC ufficiale, attività indistinguibile da un normale transfer ERC-20, no rischio legale	Privacy "solo" receiver-side	✅ Scelto

7.4 Cosa premia esattamente

Il modello attuale paga **al sender** un flat reward per ogni `sendStealthNox`. Vantaggi:

- **Semplice da farmare onesto:** chiunque voglia pagare un fornitore o donare a un creator, se passa per `NoxStealthSender`, prende il bonus.
- **Sybil-resistant ragionevolmente:** ogni tx costa gas (~\$0.01–0.10 su Base) + richiede di avere NOX da spendere. Sybilare 100k tx costa minimo \$1k di gas + capitale di NOX bloccato — e ad ogni halving il rendimento di farming dimezza.
- **Non incentiva volumi gonfiati:** è flat per tx, non proporzionale all'amount, quindi non favorisce wash-trading di importi grandi.

Limitazione nota: un sender può creare 1000 stealth address suoi e pagarli a se stesso. Però (a) costa gas, (b) ogni tx è on-chain visibile come "sender Alice ha pagato a stealth-addr", (c) il sender potrebbe scoprirsi se i suoi stealth address fanno operazioni successive collegabili.

8. Lifecycle completo

Fase 1 — Genesis open (giorno 0 → window scadenza, max 7 giorni)

```
status: !seeded, block.timestamp < closeAt
buyer: chiama mintGenesis(units), invia ETH
state: unitsSold cresce, NOX nel wallet ma intransferibili
exit:  attendi seed | attendi window | (eventualmente) refund post-grace
```

Fase 2 — Seed trigger (cap-reached o window-end)

Cap raggiunto:

```
status: unitsSold == GENESIS_CAP_UNITS
trigger: chiunque (gas-payer) chiama seedPool()
effect: pool v4 inizializzata, LP locked, MINING_SUPPLY mint al StealthMining
        contract, mintingClosed = true, transfers unlocked
```

Cap non raggiunto + window scaduta:

```
status: !seeded, block.timestamp >= closeAt, sold < cap
trigger: solo controller può chiamare seedPool()
effect: identico (pool a prezzo leggermente diverso perché meno ETH raised)
```

Nessun seed entro 48h dopo close:

```
status: !seeded, block.timestamp >= closeAt + REFUND_GRACE
trigger: ogni buyer chiama refund() singolarmente
effect: ETH restituito 1:1, NOX nel wallet restano (intransferibili per sempre)
```

Fase 3 — Trading live

```
status: seeded, mintingClosed
mercato: NOX/ETH live su Uniswap v4, prezzo determinato dal mercato
fee:     ogni swap paga 1% del NOX output al NoxHook → treasury
trasf:   liberi tra wallet, ai DEX, ovunque
```

Fase 4 — Stealth + mining attivi

```
receiver: registerKeys() una volta in StealthRegistry
sender:   approve(NoxStealthSender, amount) + sendStealthNox(...)
mining:   ogni send paga reward al sender, contatore txCount++
halving:  ogni 100k tx la reward dimezza
fine:     quando totalMined == MINING_SUPPLY (~200k tx in era 0), reward = 0
```

L'emissione del mining si esaurisce in modo geometrico. Anche dopo l'esaurimento, gli stealth payments continuano a funzionare normalmente — solo senza reward.

9. Modello di sicurezza e fiducia

9.1 Cosa è trustless

- **No admin key sui contratti.** Né `NoxToken`, né `NoxGenesis`, né `NoxHook`, né i 4 contratti stealth hanno funzioni `onlyOwner` privilegiate. Il “controller” di `NoxGenesis` ha esattamente due poteri: forzare `seedPool()` dopo che la window è scaduta, e wirare `mining.setStealthSender()` *una sola volta* dopo il deploy. Entrambi sono operazioni one-shot terminali.
- **No proxy upgradable.** Niente `UUPS`, niente `Beacon`, niente `TransparentProxy`. Il bytecode è quello deployato e non cambia mai.
- **Mint sealed.** Dopo `seedPool()`, `mintingClosed = true` e non c'è modo di rimuovere quel bit. La supply massima è raggiunta e fissata.
- **LP locked forever.** Come spiegato in §5.3.
- **Refund garantito.** Se il seed non avviene, il refund è sempre disponibile dopo 48h. Nessuno può bloccarlo.

9.2 Cosa rimane custodiale

- **Il treasury.** Riceve l'1% delle swap fee. È un EOA (o Safe multisig se preferito). Il team controlla quei fondi e può spenderli liberamente. **Non c'è impegno on-chain su come saranno usati.** Per progetti più maturi si può swappare il treasury address per uno splitter contract (DAO, vesting, operations multisig).
- **Le viewing/spending keys stealth.** Vivono nel browser dell'utente (`localStorage` nella demo) e nella sua testa. Perderle = perdere tutti i pagamenti stealth ricevuti. Non c'è recovery.
- **Off-chain availability degli Announcement events.** Il discovery dei pagamenti dipende dal poter leggere i log della chain. Su Base questo è banale (qualunque RPC), ma in scenari di censura severa potrebbe non esserlo.

9.3 Limitazioni della privacy

Cosa **non** copriamo:

- **Sender privacy.** Il sender di un `sendStealthNox` è visibile on-chain. Per nascondere anche il sender servirebbe uno shielded pool (Railgun) o un mixer (TC-style).
- **Amount privacy.** L'importo è in chiaro. Diversi sender di importi noti (esempio: stipendio fisso da una company) sono di fatto identificabili.

- **Timing analysis.** Se Alice paga sempre Bob alle 18 di lunedì, e si vede uno stealth payment alle 18 di lunedì, statisticamente è probabile sia per Bob. Diluire questa correlazione richiede o batching, o ritardo random, o relay network.
- **Network-level metadata.** L'IP del sender quando invia la transazione, gli orari di accesso al RPC, gli scan pattern del receiver. Sono tutti vettori di deanonimizzazione che il protocollo on-chain non può difendere. Soluzioni: Tor/VPN, RPC privati, scanner self-hosted.
- **Compliance “guilt-by-association”.** Se un indirizzo stealth riceve fondi che in seguito vengono ricondotti a un'attività illecita, e il receiver li spende, il *receiver* potrebbe avere problemi anche se non sapeva la provenienza. Privacy Pools (Buterin 2023) mitigano questo con association-set proofs.

10. Considerazioni legali

10.1 Il caso Pertsev e il precedente che ne deriva

Maggio 2024: Alexey Pertsev, sviluppatore olandese di Tornado Cash, è stato condannato a 5 anni 4 mesi di carcere dal tribunale di Bois-le-Duc per il suo ruolo nello sviluppo del protocollo. La condanna è in appello, ma il precedente è ora parte della giurisprudenza EU: **scrivere e deployare un mixer è, in alcune giurisdizioni europee, un reato penale**, indipendentemente dall'uso che ne fa la rete e dal fatto che il protocollo sia immutabile/non-custodial.

A novembre 2024 il 5° Circuito USA ha dichiarato illegali le sanzioni OFAC sui contratti immutabili di TC, e a gennaio 2025 OFAC ha rimosso le sanzioni — ma i procedimenti penali contro gli sviluppatori restano aperti.

10.2 Come Nox si posiziona

Il design di Nox è deliberatamente conservativo rispetto a questo rischio:

Componente	Pattern	Rischio penale comparabile
Genesis sale	Vendita di token a prezzo fisso, no security-like promises	Standard ICO/IDO, basso
Uniswap v4 pool + hook 1%	Hook fee, modello adottato da decine di token su mainnet	Standard DeFi, basso
Stealth addresses	Standard ERC ufficiale (Umbra opera dal 2020 senza incidenti)	Basso, equivalente a “inviare ERC-20 a un EOA”
Stealth Mining	Reward emessi a wallet pubblici per uso di standard ERC pubblico	Basso

Cosa abbiamo escluso esplicitamente dal design:

- ❌ Shielded pool / zk-mixer custom (= TC-clone)
- ❌ Mining reward per uso di mixer (= incentivo all'obfuscation = aggravante intent)
- ❌ Logica anti-tracing on-chain (es. ring signatures emulate)

Resta che **questo non è parere legale**. Chiunque deploia Nox in EU dovrebbe prima consultare un avvocato penalista specializzato in crypto per valutare la propria esposizione specifica.

11. Roadmap (opzionale)

Il sistema *come è* è completo e shippable. Le seguenti sono estensioni considerate ma non implementate, in ordine di priorità realistica:

11.1 Integrazione Railgun

Per chi vuole privacy *completa* (sender + receiver + amount), NOX è già pronto a essere depositato in Railgun su Base. Necessario:

- Tutorial/guida nella dApp con bottone “Shield via Railgun”.
- Integrazione SDK Railgun nel frontend.
- Eventualmente: estendere lo `StealthMining` per premiare anche le shielded transfers di Railgun (richiede un oracolo).

11.2 Privacy Pools (Buterin design 2023)

Mixer compliance-friendly: l'utente fornisce *due* zk-proof al withdraw, “appartengo a un deposito del pool” + “appartengo a un sottoinsieme di depositi puliti”. Riduce il rischio “guilt-by-association” e l'esposizione legale rispetto a un mixer classico.

Effort stimato: 3-6 mesi + audit. Già implementato da `Oxbow` su Ethereum mainnet — potenzialmente riusabile.

11.3 SDK + wallet integrations

- Pubblicare la stealth SDK come npm package separato (estraibile da `web/lib/stealth.ts`).
- Integrazione plug-in per wallet (MetaMask snap, Rabby extension).
- Indexer GraphQL/Subgraph per scanning veloce degli `Announcement` events.

11.4 Treasury splitter / DAO

Lo `treasury` può essere swappato (off-chain governance del team) verso un contratto Splitter più sofisticato: - Multisig con timelock. - DAO con voting NOX-weighted. - Vesting schedule per team comp.

12. Stato del progetto (mag 2026)

Componente	Stato	Test
NoxToken.sol	✅ Completo	✓
NoxGenesis.sol	✅ Completo	✓
NoxHook.sol	✅ Completo	✓
StealthRegistry.sol	✅ Completo	✓
StealthAnnouncer.sol	✅ Completo	✓
NoxStealthSender.sol	✅ Completo	✓
StealthMining.sol	✅ Completo	✓
Test suite	26/26 PASS	—
Deploy script Foundry	✅ Completo	—
dApp Next.js (5 pagine)	✅ Completo, build PASS	—
Stealth SDK browser	✅ Completo, build PASS	—
Audit indipendente	❌ Non ancora	Da pianificare prima di mainnet
Fork test su Base mainnet	❌ Non ancora	Step successivo
Deploy su Base Sepolia	❌ Non ancora	Dopo fork test
Deploy su Base mainnet	❌ Non ancora	Dopo audit

13. Riferimenti

Standard ERC

- [ERC-20](#) — Token standard
- [ERC-2612](#) — Permit
- [ERC-5564](#) — Stealth addresses
- [ERC-6538](#) — Stealth meta-address registry
- [ERC-6909](#) — Minimal multi-token interface (usato dal v4 PoolManager per i claim)

Uniswap v4

- [v4-core whitepaper](#)

- [v4 Hooks doc](#)
- [v4 Deployments](#)

Privacy primitives

- [Umbra Cash](#) — Prima implementazione mainstream di stealth payments
- [Fluidkey](#) — Stealth wallet con UX nativa
- [Railgun](#) — zk-shielded transfers (privacy completa, alternativa a stealth)
- [Privacy Pools design](#) — Buterin et al., 2023

Caso legale Tornado Cash

- [US DOJ indictment Storm & Semenov \(Ago 2023\)](#)
- [Sentenza Pertsev NL \(Mag 2024\)](#)
- [5° Circuito USA: contratti immutabili non sono “property” \(Nov 2024\)](#)

Riferimento di design

- [Nonce token su Basescan](#) — Il token Bitcoin-on-Base che ha ispirato il modello genesis + v4 hooks di Nox

Questo whitepaper descrive lo stato attuale del codice. Per la verità ultima, leggere il codice in `src/` e i test in `test/`. I commenti dei contratti sono normativi: in caso di discrepanza con questo documento, prevalgono i contratti.